

Milestone 2

(Capt - USMC)

Jonathan Goohs (LT - USN)

(LT - USN)

D1 - Provide an annotated output of a run indicating the place(s) where you see evidence of improper

results. [2 pts]

// START OF OUTPUT

Bob transaction for: 117, balance of: 2117

// Correct

Alice balance inquiry: 2000

// Incorrect balance, should be 2117 since Bob deposited 117.

Alice transaction for: -54, balance of: 2063

// Correct

Bob balance inquiry: 2063

// Correct

Alice transaction for: 82, balance of: 2145

// Correct

Bob balance inquiry: 2145

// Correct

Alice balance inquiry: 2145

// Correct

Alice transaction for: 261, balance of: 2406

// Correct

Bob transaction for: 233, balance of: 2378

// Incorrect balance, should be 2639.

Alice transaction for: 263, balance of: 2641

// Incorrect balance, should be 2902.

Bob transaction for: 160, balance of: 2538

// Incorrect balance, should be 3062.

Bob transaction for: 51, balance of: 2589

// Incorrect balance, should be 3113.

Bob transaction for: -4, balance of: 2585

// Incorrect balance, should be 3109.

Alice transaction for: -31, balance of: 2558

// Incorrect balance, should be 3078.

Bob balance inquiry: 2558

// Incorrect balance, should be 3078.

Bob transaction for: 171, balance of: 2729

// Incorrect balance, should be 3249.

Alice transaction for: 296, balance of: 2854

// Incorrect balance, should be 3545.

Alice balance inquiry: 2854

// Incorrect balance, should be 3545.

Bob transaction for: -225, balance of: 2629

// Incorrect balance, should be 3320.

Alice transaction for: 219, balance of: 2848

// Incorrect balance, should be 3539.

Bob transaction for: -269, balance of: 2579

// Incorrect balance, should be 3270.

Alice balance inquiry: 2579

ATMServer Bob shut down

// Incorrect balance, should be 3270.

Bob transaction for: 137, balance of: 2716

ATM machine Bob shutting down; transaction total was: 371

// Incorrect balance, should be 3407.

Alice transaction for: 2, balance of: 2581

ATMServer Alice shut down

ATM machine Alice shutting down; transaction total was: 1038

// Incorrect balance, should be 3409.

Final account total is 2581

// Incorrect balance, should be 3409.

Done with ATMs!

// END OF OUTPUT

D2 - In isolation, for a single customer at a time. [4 pts]

a) Is the Transaction Protocol from the above figure correct?

Bob transaction for: 17, balance of: 2017

Bob transaction for: -129, balance of: 1888

Bob transaction for: 11, balance of: 1899

Bob transaction for: 202, balance of: 2101

Bob transaction for: 299, balance of: 2400

Bob transaction for: 115, balance of: 2515

Bob transaction for: -53, balance of: 2462

Bob transaction for: -71, balance of: 2391

Bob transaction for: 25, balance of: 2416

Bob transaction for: 274, balance of: 2690

ATM machine Bob shutting down; transaction total was: 690

Yes, in isolation single customer iterations of the code run without issues, suggesting there are issues with the shared resource.

b) Justify

The transaction protocol, as implicitly depicted in Parallelism.pdf and implemented in the ATM client, involves a sequence of steps:

1. Get Balance: Retrieve the current account balance from the bank database.
2. Modify Locally: Add or subtract the transaction amount to/from the retrieved balance.
3. Put Balance: Write the new calculated balance back to the bank database.

When only a single customer (process) is accessing the account, there is no other process that can interfere with this sequence of operations. The "step-by-step behavior" and "one thing at a time" nature of sequential processes would ensure that the balance retrieved is always the most current, the calculation is performed without interruption, and the result is written back without being overwritten by a concurrent operation. In this scenario, the full transaction as described above as an indivisible action. The issue of "race conditions," where the output depends on the speeds or interleaving of parallel paths, does not arise because there is no parallelism involving shared resources.

D3 - With respect to D1 & D2 parts d) [8 pts]

a) What do you believe the problem is?

We believe the fundamental problem is a race condition caused by the lack of mutual exclusion during the critical section of the read-modify-write cycle transaction acting on the account balance. The system allows multiple concurrent processes (ATM clients and their associated ATM servers) to access and modify the same shared account data without adequately ensuring that each complete transaction (getting the balance, modifying it, and putting it back) occurs in proper succession. As a result, operations from different clients

can interleave, leading to lost updates and an inconsistent final balance as we see in the above output of the script.

b) Where in the system is the problem manifested? Be specific down to lines of code and the simulation action relationships you identify as direct contributors.

The problem is manifested across both the ATM client (ATM.py) and the ATM server (ATMServer.py), due to their interaction and the server's lack of proper synchronization for multi-step transactions. The simulation action relationships directly highlight these issues through observable incorrect balances.

****ATM.py (ATM Client):**** The problem begins with the client's transactional sequence, which involves multiple steps that are not treated as a single operation by the server. Each ATM client performs a transaction by:

- Sending a GET_BALANCE request
- Receiving the current balance from the server
- Calculating the new balance locally (balance += transactionAmount), creating a local copy that can become stale if another transaction modifies the actual balance on the server
- Sending a PUT_BALANCE request with the updated balance

This separation of the "read" (GET_BALANCE) and "write" (PUT_BALANCE) operations, with local modification in between, creates the window for a race condition.

****ATMServer.py (ATM Server):**** The server's design for handling these requests is where the race condition becomes critical:

- The server handles GET_BALANCE and PUT_BALANCE as two entirely separate and independent operations within its execute method:
- GET_BALANCE calls self.OS.read(self.account) via getBalance()
- PUT_BALANCE calls self.OS.write(self.account, newBalance) via putBalance()

- There is no synchronization between these operations for a single client's transaction
- After one ATM client sends GET_BALANCE and before it sends PUT_BALANCE, other ATM servers can interleave their own operations on the same shared account data
- When two ATMs read the balance simultaneously, both see the same initial value, calculate new balances locally, and send PUT_BALANCE requests. The second PUT_BALANCE overwrites the first, causing lost transactions as seen in the output above.

-----EOF-----